



HACKTHEBOX



Questionnaire

7th October 2022 / Document No.
D22.102.253

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: **Very Easy**

Classification: Official

Synopsis

Questionnaire is a very easy difficulty challenge that teaches the basics of C, protections of a binary and Buffer Overflows

Skills Required

- N/A

Skills Learned

- Basic C syntax
- Buffer Overflow

Enumeration

The given downloadable contains 2 files:

- `test.c`
- `test`

Taking a look at `test.c`

```
#include <stdio.h>
#include <stdlib.h>

/*
This is not the challenge, just a template to answer the questions.
To get the flag, answer the questions.
There is no bug in the questionnaire.
*/
```

```
void gg() {
    system("cat flag.txt");
}

void vuln() {
    char buffer[0x20] = {0};
    fprintf(stdout, "\nEnter payload here: ");
    fgets(buffer, 0x100, stdin);
}

void main() {
    vuln();
}
```

This is not the binary we have to pwn. Actually, there is no binary to pwn at all this time. All the users have to do is answer the questions to get the flag. Connecting to the remote instance:

This is a simple questionnaire to get started with the basics.

```
.....
•
• When compiling C/C++ source code in Linux, an ELF (Executable and Linkable Format) file is created.
• The flags added when compiling can affect the binary in various ways, like the protections.
• Another thing affected can be the architecture and the way it's linked.
•
• If the system in which the challenge is compiled is x86_64 and no flag is specified,
• the ELF would be x86-64 / 64-bit. If it's compiled with a flag to indicate the system,
• it can be x86 / 32-bit binary.
•
• To reduce its size and make debugging more difficult, the binary can be stripped or not stripped.
•
• Dynamic linking:
•
• A pointer to the linked file is included in the executable, and the file contents are not included
• at link time. These files are used when the program is run.
•
• Static linking:
•
• The code for all the routines called by your program becomes part of the executable file.
•
• Stripped:
•
• The binary does not contain debugging information.
•
• Not Stripped:
•
• The binary contains debugging information.
•
• The most common protections in a binary are:
•
• Canary: A random value that is generated, put on the stack, and checked before that function is
• left again. If the canary value is not correct-has been changed or overwritten, the application will
• immediately stop.
•
• NX: Stands for non-executable segments, meaning we cannot write and execute code on the stack.
•
• PIE: Stands for Position Independent Executable, which randomizes the base address of the binary
• as it tells the loader which virtual address it should use.
•
• RelRO: Stands for Relocation Read-Only. The headers of the binary are marked as read-only.
•
• Run the 'file' command in the terminal and 'checksec' inside the debugger.
•
• The output of 'file' command:
•
• x file test
• test: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked,
• interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=5a83587fbd6ad7b1aeee2d59f027a882bf2a429,
• for GNU/Linux 3.2.0, not stripped.
•
• The output of 'checksec' command:
•
• gef> checksec
• Canary                : X
• NX                    : ✓
• PIE                   : X
• Fortify               : X
• RelRO                 : Partial
•
•.....

[*] Question number 0x1:

Is this a '32-bit' or '64-bit' ELF? (e.g. 1337-bit)

>>
```

This is a simple questionnaire that teaches some basic stuff and then asks the user to answer some questions.

All the questions are about the `test` binary. As we can see, it is a `64-bit` ELF. Once we answer correctly, we proceed to the next question.

```

[*] Question number 0x1:
Is this a '32-bit' or '64-bit' ELF? (e.g. 1337-bit)
>> 64-bit
  ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲
  ▲       Correct       ▲
  ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲
  ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲

[*] Question number 0x2:
What's the linking of the binary? (e.g. static, dynamic)
>>

```

After we answer correctly, we are prompted to the next question. If the answer is wrong, the question is repeated.

```

>> dynamic
  ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲
  ▲       Correct       ▲
  ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲
  ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲

[*] Question number 0x3:
Is the binary 'stripped' or 'not stripped'?
>> stripped
  ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲
  ▲       Wrong        ▲
  ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲
  ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲

[*] Question number 0x3:
Is the binary 'stripped' or 'not stripped'?
>>

```

There are 3 sections that teach the users and 10 question in total. All of the questions can be answered just from the questionnaire itself and the `test.c`.

```

This is a simple questionnaire to get started with the basics.

.....
•
• When compiling C/C++ source code in Linux, an ELF (Executable and Linkable Format) file is created.
• The flags added when compiling can affect the binary in various ways, like the protections.
• Another thing affected can be the architecture and the way it's linked.
•
• If the system in which the challenge is compiled is x86_64 and no flag is specified,
• the ELF would be x86-64 / 64-bit. If it's compiled with a flag to indicate the system,
• it can be x86 / 32-bit binary.
•
• To reduce its size and make debugging more difficult, the binary can be stripped or not stripped.
•
• Dynamic linking:
•
• A pointer to the linked file is included in the executable, and the file contents are not included
• at link time. These files are used when the program is run.
•
• Static linking:
•
• The code for all the routines called by your program becomes part of the executable file.
•
• Stripped:
•
• The binary does not contain debugging information.
•
• Not Stripped:
•
• The binary contains debugging information.
•
• The most common protections in a binary are:
•
• Canary: A random value that is generated, put on the stack, and checked before that function is
• left again. If the canary value is not correct-has been changed or overwritten, the application will
• immediately stop.
•
• NX: Stands for non-executable segments, meaning we cannot write and execute code on the stack.
•
• PIE: Stands for Position Independent Executable, which randomizes the base address of the binary
• as it tells the loader which virtual address it should use.
•
• RelRO: Stands for Relocation Read-Only. The headers of the binary are marked as read-only.
•
• Run the 'file' command in the terminal and 'checksec' inside the debugger.
•
• The output of 'file' command:
•
• x file test
• test: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked,
• interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=5a83587fbda6ad7b1aeed2d59f027a882bf2a429,
• for GNU/Linux 3.2.0, not stripped.
•
• The output of 'checksec' command:
•
• gef> checksec
• Canary                : ✖
• NX                    : ✔
• PIE                   : ✖
• Fortify               : ✖
• RelRO                 : Partial
•
.....

[*] Question number 0x1:

Is this a '32-bit' or '64-bit' ELF? (e.g. 1337-bit)

>> 64-bit

▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲
▲
▲      Correct      ▲
▲
▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲

[*] Question number 0x2:

What's the linking of the binary? (e.g. static, dynamic)

>> dynamic

▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲
▲
▲      Correct      ▲
▲
▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲

[*] Question number 0x3:

```

Is the binary 'stripped' or 'not stripped'?

>> not stripped

```
▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲
▲           ▲
▲   Correct   ▲
▲           ▲
▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲
```

[*] Question number 0x4:

Which protections are enabled (Canary, NX, PIE, Fortify)?

>> nx

```
▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲
▲           ▲
▲   Correct   ▲
▲           ▲
▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲
```

```

Great job so far! Now it's time to see some C code and a binary file.

In the pwn_questionnaire.zip there are two files:

1. test.c
2. test

The 'test.c' is the source code and 'test' is the output binary.

Let's start by analyzing the code.
First of all, let's focus on the '#include <stdio.h>' line.
It includes the 'stdio.h' header file to use some of the standard functions like 'printf()'.
The same principle applies for the '#include <stdlib.h>' line, for other functions like 'system()'.

Now, let's take a closer look at:

void main(){
    vuln();
}

By default, a binary file starts executing from the 'main()' function.

In this case, 'main()' only calls another function, 'vuln()'.
The function 'vuln()' has 3 lines.

void vuln(){
    char buffer[0x20] = {0};
    fprintf(stdout, "\nEnter payload here: ");
    fgets(buffer, 0x100, stdin);
}

The first line declares a 0x20-byte buffer of characters and fills it with zeros.
The second line calls 'fprintf()' to print a message to stdout.
Finally, the third line calls 'fgets()' to read 0x100 bytes from stdin and store them to the
aforementioned buffer.

Then, there is a custom 'gg()' function which calls the standard 'system()' function to print the
flag. This function is never called by default.

void gg(){
    system("cat flag.txt");
}

Run the 'man <function_name>' command to see the manual page of a standard function (e.g. man fgets).

```

[*] Question number 0x5:

What is the name of the custom function the gets called inside `main()`? (e.g. vulnerable_function())

```
>> vuln
```

```

  * * * * *
  *       *
  *   Correct   *
  *       *
  * * * * *

```

[*] Question number 0x6:

What is the size of the 'buffer' (in hex or decimal)?

```
>> 32
```

```

  * * * * *
  *       *
  *   Correct   *
  *       *
  * * * * *

```

[*] Question number 0x7:

Which custom function is never called? (e.g. vuln())

```
>> gg
```

```

  * * * * *
  *       *
  *   Correct   *
  *       *
  * * * * *

```

```

Excellent! Now it's time to talk about Buffer Overflows.

Buffer Overflow means there is a buffer of characters, integers or any other type of variables,
and someone inserts into this buffer more bytes than it can store.

If the user inserts more bytes than the buffer's size, they will be stored somewhere in the memory
after the address of the buffer, overwriting important addresses for the flow of the program.
This, in most cases, will make the program crash.

When a function is called, the program knows where to return because of the 'return address'. If the
player overwrites this address, they can redirect the flow of the program wherever they want.
To print a function's address, run 'p <function_name>' inside 'gdb'. (e.g. p main)

gef> p gg
$1 = {<text variable, no debug info>} 0x401176 <gg>

To perform a Buffer Overflow in the simplest way, we take these things into consideration.

1. Canary is disabled so it won't quit after the canary address is overwritten.
2. PIE is disabled so the addresses of the binary functions are not randomized and the user knows
   where to return after overwriting the return address.
3. There is a buffer with N size.
4. There is a function that reads to this buffer more than N bytes.

Run printf 'A%.0s' {1..30} | ./test to enter 30*A" into the program.

Run the program manually with "./test" and insert 30*A, then 39, then 40 and see what happens.

```

```

[*] Question number 0x8:

What is the name of the standard function that could trigger a Buffer Overflow? (e.g. fprintf())

>> fgets

  ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲
  ▲      Correct      ▲
  ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲

[*] Question number 0x9:

Insert 30, then 39, then 40 'A's in the program and see the output.

After how many bytes a Segmentation Fault occurs (in hex or decimal)?

>> 40

  ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲
  ▲      Correct      ▲
  ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲

[*] Question number 0xa:

What is the address of 'gg()' in hex? (e.g. 0x401337)

>> 0x401176

  ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲
  ▲      Correct      ▲
  ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲ ▲

Great job! It's high time you solved your first challenge! Here is the flag!

HTB{f4k3_fl4g_4_t35t1ng}

```